Cairo University
Faculty of Engineering



CMPS458 – Reinforcement Learning

Assignment 2: Policy Iteration

By: Rana Ahmed Mohamed Samir Elgharabawy
code: 1230031
GitHub Repo link: https://github.com/RanaElgharabawy/RL_GridMaze

Date: 1/11/2026

A) Outcome Summary:

1. Environment. Implemented GridMazeEnv, a custom Gymnasium environment (gym.Env) for the 5×5 Grid Maze:

- On every reset(), the agent start, the goal G and the two bad cells X are placed randomly on 4 distinct cells using Gymnasium's seeded random generator (reproducible with a seed).
- Observation space: 8 numbers: the positions of the agent, the goal and the two bad cells. x is the column and y the row, with (0,0) the top-left cell.
- Action space: Discrete(4) with 0 = right, 1 = up, 2 = left, 3 = down. Moves are deterministic; moving into the border leaves the agent in place.
- Rewards: +1 for reaching G, −1 for reaching X (both end the episode), 0 otherwise. An episode is truncated after 100 steps, which only matters if the goal is completely enclosed by bad cells.
- Rendering with PyGame in two modes: human (live window) and rgb_array (image frames for video recording). The goal is drawn green with "G", bad cells red with "X", and the agent as a yellow circle.
- The environment passes Gymnasium's official check_env validator.

2. Policy Iteration. Since the goal and bad cells are fixed within an episode, the planning state is the agent's position (25 states). The environment provides the model P[s][a] = (next_state, reward, terminated) for the current maze. Policy iteration starts from the policy "always move right" and alternates:

- **Policy evaluation:** in-place Bellman expectation updates, $V(s) = r + \gamma V(s')$, with $\gamma = 0.99$, until the largest change is below $\theta = 10^{-6}$.
- **Policy improvement:** each state switches to the action with the highest $Q(s,a) = r + \gamma V(s')$, only when it is strictly better than the current action, to avoid oscillation between equally good actions.

The loop stops when no state changes its action. Policy iteration is re-run after every reset, because each new maze is a new MDP.

3. Results.

- Over 200 random mazes, policy iteration converged in **6.07 iterations on average** (min 3, max 9), and the resulting policy reached the goal in **200/200** mazes.
- In the 10 recorded episodes (seeds 0–9) the agent reached the goal every time (return +1) in 2–6 steps, always along a shortest path that avoids the bad cells.
- The learned value function decreases by a factor of $\gamma = 0.99$ per step away from the goal, so the policy follows shortest paths.

4. Videos. The agent was recorded with Gymnasium's RecordVideo wrapper, which saved ten episodes as videos/grid_maze-episode-0.mp4 to videos/grid_maze-episode-9.mp4, recorded at 10 fps with every frame repeated 5 times, so each move is shown for 0.5 s and the videos play correctly in any player.

5. Repository: https://github.com/RanaElgharabawy/RL_GridMaze

B) Answers to the Questions:

Q1. What is the state-space size of the 5x5 Grid Maze problem?

The answer depends on what we count as the "state":

- **Full observation space:** the observation is 8 integers, each in {0,…,4}, so the observation space contains $5^8 = 390{,}625$ vectors (25^4, since there are 4 positions on 25 cells).
- **Valid maze configurations:** the agent, goal and two bad cells must occupy 4 distinct cells, giving $25 \times 24 \times 23 \times 22 = 303{,}600$ ordered arrangements. The two bad cells are interchangeable, so there are 151,800 distinct configurations.
- **State space used by policy iteration:** within one episode the goal and bad cells are fixed, so the only thing that changes is the agent's position. The planning problem therefore has 25 states, of which 3 are terminal (the goal and the 2 bad cells), leaving **22 non-terminal states.** Our implementation solves this 25-state MDP once per generated maze.

Q2. How to optimize the policy iteration for the Grid Maze problem?

Several optimizations apply:

1. **Reduce the state space:** plan over the 25 agent positions for the current maze instead of the full 8-integer observation (390,625 vectors). This is the largest saving.
2. **In-place (Gauss-Seidel) evaluation:** each updated V(s) is used immediately in the same sweep instead of waiting for the next sweep, so values propagate faster and fewer sweeps are needed.
3. **Stable tie-breaking:** the policy changes an action only if another action is strictly better. This prevents oscillation between equally good actions, which would otherwise stop the algorithm from terminating.
4. **Modified (truncated) policy iteration:** run only k evaluation sweeps (e.g. k = 3–5) per iteration instead of evaluating to full convergence. The extreme case k = 1 is value iteration.
5. **Better sweep order / initial policy:** sweep states starting from those nearest the goal, or initialize the policy greedily towards the goal, so fewer iterations are needed.

Optimizations 1-3 are used in the implementation.

Q3. How many iterations did it take to converge on a stable policy for 5x5 maze?

The number of iterations depends on the randomly generated maze (where the goal and bad cells are) and on the initial policy (we start from "always move right"), so we measured it over many mazes:

Experiment	Policy-iteration iterations
Example maze (seed 42)	**7** iterations (18 evaluation sweeps in total)
10 recorded episodes (seeds 0–9)	between **3 and 8**
200 random mazes (seeds 0–199)	mean **6.07**, min **3**, max **9**

In all 200 mazes the resulting policy reached the goal, as did the agent in all 10 recorded episodes (return +1 in 2–6 steps). Policy iteration therefore converges in a small number of iterations, typically about 6 on a 5×5 maze. This makes sense because each iteration fixes the moves of cells one step further from the goal, so the farthest cell from the goal decides roughly how many iterations are needed.

Q4. Explain, with an example, how policy iteration behaves with multiple goal cells.

Policy iteration needs no modification for multiple goals: each goal is simply another terminal state with reward +1. Since every state chooses the action with the highest discounted return, and a goal reached in k steps is worth γ^(k−1), each state is directed to the goal it can reach in the fewest safe steps. The maze is divided into "basins of attraction", one per goal (Each goal pulls the cells closest to it, so the maze splits into two areas.).

Example: a 5×5 maze with goals at (0,0) and (4,4) and bad cells at (2,1) and (2,3), with γ = 0.99. Policy iteration converged in 5 iterations, the same as a single-goal maze:

```
(venv) PS C:\Users\STW\Desktop\RL_assignment2> python experiments.py
=== Q4: two goals at (0,0) and (4,4), bad cells at (2,1) and (2,3) ===
converged in 5 iterations
G < < < <    0.000  1.000  0.990  0.980  0.970
^ ^ X v v    1.000  0.990  0.000  0.970  0.980
^ ^ > v v    0.990  0.980  0.970  0.980  0.990
^ v X v v    0.980  0.970  0.000  0.990  1.000
> > > > G    0.970  0.980  0.990  1.000  0.000
```

Observations:

- **Two regions:** cells in the top-left move towards G(0,0), while cells on the right and bottom move towards G(4,4). The value function has two peaks, one next to each goal, decreasing by a factor of 0.99 per step away.
- **Obstacles reshape the regions:** the bad cell at (2,1) blocks the direct route, so the top row moves left towards (0,0), while the cells below the bad cell go down towards (4,4).
- **Ties:** cells that are equally far from both goals, such as the centre (2,2) and (1,3), have the same value (0.970) for both choices. Both actions are optimal, and our tie-breaking rule keeps the first one found.
- If the goals had **different rewards** (e.g. +1 and +2), a state would choose the farther goal whenever its discounted reward γ^(k−1)r is larger, so the basin/region of the more valuable goal grows.

Q5. Can policy iteration work on a 100x100 maze? Explain why?

Yes, as long as the goal and bad cells are fixed and the state is the agent's position. The maze then has 100 × 100 = 10,000 states and 4 actions. That's still a small tabular MDP: the value table and transition model fit in memory, and policy iteration is still guaranteed to converge to the optimal policy.

We measured the runtime of our (pure Python) implementation on growing mazes:

```
=== Q5: scaling (goal in one corner, 2 bad cells, agent anywhere) ===
    size   states  iterations   sweeps  time (s)
    5x5        25           6       47      0.00
  10x10       100          10      145      0.01
  20x20       400          20      590      0.16
  30x30       900          30     1335      0.77
  50x50      2500          50     3725      7.12
 100x100    10000         100    14950     96.47
```

The 100×100 maze was solved, but it shows the method's main cost: the number of iterations grows linearly with the maze width n, the sweeps per evaluation also grow with n, and each sweep visits all n^2 states. The total work grows roughly as $O(n^4)$: doubling the maze from 50×50 to 100×100 made it about 14× slower.

However, if the goal and bad-cell positions are also part of the state (as in the full observation), the state space becomes $10{,}000^4 = 10^{16}$ states. A value table that large cannot be stored, let alone swept repeatedly. This is the curse of dimensionality, and it's a fundamental limitation of dynamic programming, which must visit every state on every sweep.

Q6. Can policy iteration work on a continuous-space maze? Explain why?

Not directly. Classical policy iteration is a tabular method: it stores one value V(s) and one action π(s) for every state, and each evaluation sweep loops over all states. In a continuous maze the agent's position (x, y) is a real-valued vector, so there are infinitely many states:

- V and π can't be stored as tables, since there are infinitely many entries.
- A sweep "for every state s" never finishes.

Policy iteration can only be applied approximately:

1. Discretization: divide the continuous space into a grid of cells and solve the resulting finite MDP. This is simple, but it introduces approximation error (two physically different positions in one cell are treated as the same state), and the number of cells grows exponentially with the number of dimensions (curse of dimensionality). A finer grid is more accurate but slower, as Q5 showed.
2. Function approximation: represent V(s) or π(s) with a parameterized function (linear features or a neural network) and fit it to Bellman targets computed at sampled states (approximate / fitted policy iteration). This is the basis of modern methods such as actor-critic and deep RL, which no longer require a full model or a sweep over all states.

Q7. Can policy iteration work with moving bad cells (like Packman moving ghosts)? Explain why?

It depends on whether the ghosts' movement is known and on how large the resulting state space becomes.

Why our current approach fails: our policy iteration plans with the agent's position as the only state, assuming the bad cells are fixed. If the bad cells move, the same agent position can be safe at one moment and deadly the next, so a policy that depends only on the agent's position can't be optimal. The future depends on information (the ghost positions) that isn't in the state.

When it can work: policy iteration still applies if:

1. The ghost positions are added to the state, so the state becomes (agent, $ghost_1$, $ghost_2$). On a 5×5 grid this is 25 × 25 × 25 = 15,625 states (with a fixed goal), which is still solvable.
2. The ghosts' movement model is known and depends only on the current state, e.g. "each ghost moves randomly to a neighbouring cell", or "each ghost moves towards the agent with probability 0.8". The transitions then become stochastic, $P(s' \mid s, a)$, with a sum over all possible ghost moves, which the Bellman equation handles directly.

When it fails or becomes impractical:

- **Unknown ghost behaviour:** policy iteration requires a full model of the dynamics. If we don't know how the ghosts move, we can't build $P(s' \mid s, a)$, and we need model-free methods that learn from experience.
- **Large state spaces:** the state space grows exponentially with the number of ghosts: (cells)^(ghosts + 1). Real Pac-Man has far too many states to sweep, the same curse of dimensionality as in Q5.